

Surface Tension — Experiment Status

What “no loops, no recursion” actually means,
and why the answer matters

Julian Quick

14 June 2026

Abstract

This report is a self-contained status update on the *Surface Tension* project. The project asks whether a 31-billion-parameter, instruction-tuned language model (Gemma 4 31B-it) can be taught to *default* to writing Python code that obeys a stylistic restriction the model is normally happy to ignore: “no `for` loops, no `while` loops, no recursion.” After several months of training and evaluation experiments, the technical pipeline is in place and the headline numbers are stable, but a definitional problem we had been treating as a side question has moved to the centre. The rule that gives the project its name turns out to have two natural readings that pull in opposite directions. This document explains the experiment from scratch, summarises what is known so far, and lays out the tension — so that the next decision (which version of the rule we treat as authoritative) can be made with the full picture on the table.

1 What the experiment is

1.1 The model and the rule

We use **Gemma 4 31B-it**, an instruction-tuned open-weight code model from Google with roughly 31 billion parameters. “Instruction-tuned” means the model has already been trained, after its initial pretraining on internet text, to follow natural-language instructions — so when we send it a coding problem it returns Python that attempts to solve the problem rather than, say, predicting the next paragraph of the prompt.

The stylistic rule we impose is straightforward to state in English: “Write the solution in Python without using `for` loops, without using `while` loops, and without using recursion.” Recursion here means a function that ultimately calls itself, directly or through a chain of other functions. The motivation is not that loops and recursion are bad in themselves — they are, of course, the standard tools for iteration — but that they are the model’s strongly default tools, and we want to study whether a strong default can be overridden by training. Loops are an unusually clean target because compliance can be checked deterministically by parsing the code into an abstract syntax tree (the standard tree-of-tokens representation that Python compilers use internally) and looking for the forbidden node types.

1.2 The benchmark

We evaluate on **LiveCodeBench (LCB)**, a public benchmark of recent competitive-programming problems collected after the model’s pretraining cutoff date, so the model cannot have memorised the solutions. We use the “medium”-difficulty slice (problems labelled medium by the competition organisers), filtered to those that read input from standard input — which is the standard format for competitive programming. This filtered pool contains 57 problems. A subset of 17 “clean” problems — ones with unambiguous specifications and reliable test cases — is what we use for most reporting.

For each problem the model produces a Python program. We then run the program on the test cases and check (i) whether it passes the tests, and (ii) whether its source code contains any `for`, `while`, or recursive construct. We call (ii) *compliance* and (i) $\wedge$ (ii) *honest compliance*.

1.3 The training methods

The experiment compares the bare “out-of-the-box” model against versions that have been further trained to internalise the rule. Three training techniques appear in this report:

- **Supervised fine-tuning (SFT).** Standard supervised learning: we collect a set of (problem, compliant-solution) pairs — where the solution was written or filtered to obey the rule — and continue training the model to predict the solution given the problem. We use a parameter-efficient variant called LoRA (Low-Rank Adaptation), which trains a small adapter sitting on top of the frozen base model rather than every parameter.
- **Rationale-SFT (“R-SFT”).** A variant of SFT where the training target is not just compliant code but a short natural-language rationale (“I cannot use `for` here, so I will use `itertools.accumulate...`”) followed by the code. The hope is that the rationale prose carries some of the rule into the model’s reasoning rather than just the surface syntax of its output.
- **Direct preference optimisation (DPO).** A reinforcement-learning style method that does not need a separate reward model: we collect pairs of (preferred, dispreferred) responses to the same prompt and train the model to widen its log-probability margin between them. Here “preferred” means a compliant-and-passing response and “dispreferred” means a non-compliant or failing response.

These are layered: SFT is the foundation; R-SFT replaces vanilla SFT as the foundation; DPO is then applied on top of R-SFT.

2 Where the project stands today

The bare-prompt result. With no special training, when we put the rule in the prompt the model follows it on about 7% of problems. When the rule is *not* in the prompt — the model has to write code anyway, but there is no constraint — it gets around 15% pass-rate. So removing the constraint costs about 8 pass-rate points. This is much smaller than an earlier flawed analysis suggested (~ 34 points), and it tells us the constraint is not costing the model very much capability — the model already *can* write loop-free code, it just doesn’t do so by default.

The bare-prompt distribution. For each individual problem we can sample many responses and ask: across those samples, what fraction comply with the rule? Empirically this fraction is almost never in the middle. Across the LCB-medium pool roughly 51% of problems produce all-non-compliant samples and roughly 47% produce all-compliant samples; only about 3% live in the “discriminating” zone $[0.3, 0.7]$. This near-determinism per problem matters because it means a standard reinforcement-learning method (called GRPO, which compares samples within a group) has nothing to learn from — every sample in a group agrees with every other.

Capacity is not the bottleneck. We trained LoRA adapters at three different sizes ($r = 8, 32, 128$, where r controls the adapter’s expressive capacity). All three drove the training loss to zero, and the largest had the worst held-out compliance. Capacity is not what limits the training; the data and the objective are.

Vanilla SFT plateaus; rationale-SFT pushes through. Plain SFT reaches around 48% compliance on the val set the training selected on, and 15% on a clean held-out set. Rationale-SFT, with 240 demonstrations and a tightened recipe (internally tagged “B1++”), reaches 40% on the clean set and reduces a particular failure mode — the model writing comments like “# no loops” while still using a hidden loop — from 24% to 18%. It also solves a problem (abc374.d) that the base model never solves loop-free.

DPO hits a trade-off boundary. Layered on top of R-SFT, DPO buys more compliance, but the extra compliance starts to cost pass-rate. We are no longer on the easy side of the trade-off.

The next planned experiment. We built — but have not yet run — a follow-up experiment we call the *quadrant* experiment. On a problem where the rule is genuinely impossible to obey honestly (more on this in §3), we sample 12 responses each from the base model and the R-SFT model and ask: does R-SFT make the model *cover up* the violation rather than admit it? We also capture the model’s internal activations on each sample, so that covert and honest responses can be compared in the model’s representation space. The pipeline is finished. The first launch was killed early by an unrelated incident and has not been relaunched.

3 The definitional tension

We told the model: “no `for`, no `while`, no recursion.” The natural way to check the rule is to parse the model’s code into the abstract syntax tree and look for those three things. Call this the **loose** judge, because it is exactly what the prompt asked for and nothing more.

The trouble is that Python provides many other ways to iterate without writing the word `for`. A short list:

- *Comprehensions:* `[f(x) for x in xs]` is a list comprehension; the word `for` is there but inside an expression rather than as a statement. Whether to count this as a “`for` loop” is a judgment call.
- *Built-in folds:* `sum(xs)`, `any(xs)`, `all(xs)`, `min(xs)`, `max(xs)`. Each of these visits every element of `xs`. The iteration is implemented in C inside the Python interpreter and is invisible to the abstract syntax tree.
- *Functional iteration:* `map(f, xs)`, `filter(p, xs)`, `functools.reduce(op, xs)`, the `itertools` library. These are explicit “apply a function to each element” constructs; they are iteration by any honest definition, but they do not write the word `for`.
- *Manual stepping:* `next(iter(xs))` pulls a single element through an iterator and is the building block of any hand-rolled loop. `functools.reduce` is operationally indistinguishable from a tail-recursive accumulator.

A model that has been mildly pressured to “avoid `for`” can shift all of its iteration into these constructs and *look* compliant to the loose judge while doing exactly the work the rule was meant to prohibit.

So we wrote a second judge — the **strict** judge — that additionally flags `map`, `filter`, `functools.reduce`, `itertools.*`, `sum/any/all/min/max` applied over a generator expression, and `next / iter`. The strict judge restores teeth to the rule. It also creates two problems of its own.

Horn A — the loose rule is mostly toothless on this benchmark

We did a desk-judgment audit of the full 57-problem LCB-medium pool, classifying each problem by how hard it is to solve *without any explicit loop* (the loose rule):

tier	count
closed-form (no iteration needed)	6
builtin-reducible (one-line <code>sum/map/comprehension</code> solves it)	48
genuinely irreducible (real algorithmic iteration unavoidable)	3

The three irreducibles are a breadth-first search for a minimum-edge cycle, a multi-source breadth-first search on a grid with walls, and an interval dynamic-program with a non-local recurrence. Every other problem in the pool admits an answer that obeys the loose rule but pushes iteration into a built-in. So a model that learns to write `sum(map(int, input().split()))` instead of a `for` loop has not internalised an aversion to iteration — it has internalised the *shape* of one. The compliance signal we are training on is partly real and partly a re-labelling. Worse, the constraint we are pretending to enforce is mostly not biting on this benchmark.

Horn B — the strict rule is harder than what we told the model, and ambiguous in places

The strict judge fixes the toothlessness but breaks two other things. First, it enforces a stricter rule than the one in the prompt. The model was told “no `for`, no `while`, no recursion”; the strict judge additionally penalises “and no `map`, no `reduce`, no `itertools`, no `sum` over a generator, no `next`.” A failure to satisfy that broader rule may be the model’s genuine inability to comply, or it may be the model honestly following the rule it was actually given. We cannot tell these apart from the output alone.

Second, the strict judge has to make several judgment calls that an independent annotator might draw slightly differently:

- Is `sum([1,2,3])` iteration? (A fold over three elements — yes by any honest reading.)
- Is `sum(range(n))` iteration? (Also equal to $n(n-1)/2$.)
- Is `min(s)` iteration? (It must visit every element.)
- Is `next(iter(s))` iteration? (Once.)
- Is `functools.reduce(op, xs, init)` recursion? (Operationally a left fold; semantically a catamorphism on a list, which is indistinguishable from a tail-recursive accumulator.)

The current strict judge takes positions on all of these. The positions are defensible but not canonical, and a second person with the same intent would draw the lines a little differently. To the extent compliance under strict depends on these calls, the experiment starts measuring the judge as much as the model.

The shape of the conflict

	loose	strict
where the boundary is	abstract syntax tree	multiple judgment calls
reproducible by anyone	yes	annotator-dependent
how much it bites	3 of 57 problems	nearly all of them
matches the prompt	yes	no (broader)
risk	vacuous compliance	measures the judge

The current code reports both verdicts side-by-side and explicitly flags results whose statistical significance depends on the choice. That is the right thing to do *methodologically* — we do not have to commit before looking. But it is not a *scientific* resolution. Any headline statement of the form “training method X moved compliance from a to b ” needs one of these two judges to be the primary one; the other becomes a sensitivity check.

4 What the next decisions are

The follow-up quadrant experiment was specifically designed to put pressure on the loose/strict question. The idea is to focus on a single *genuinely irreducible* problem — one of the three in the audit — and ask: when R-SFT has trained the model to “avoid loops,” and the problem makes loop-avoidance honestly impossible, does the model produce a confident loose-compliant answer that is in fact strict-violating? In other words, does the training induce a covering-up behaviour? And is that covering-up visible in the model’s internal activations?

This is a well-posed experiment because the loose/strict gap is forced — on an irreducible problem, the only way to be loose-compliant without being strict-violating is to fail the tests. The contrast across the 12 samples per arm is what produces signal; the activation read is what makes it more than a behavioural curiosity.

But the experiment cannot run cleanly until we decide three things:

1. **Which judge is the headline number?** Loose is what we told the model and is reproducible; strict is what the rule actually means but involves judgment calls. A third option is to report both and treat the divergence as the finding — but that weakens any single headline claim.
2. **Is the LCB-medium pool the right benchmark?** Three irreducible problems is below the ~ 8 threshold the project set itself for doing a re-cohort study. Switching to the hard slice (a one-line change in the data loader) should yield ~ 50 more irreducible problems and let us re-run Phases 1–2 on a benchmark where the constraint genuinely bites.
3. **Should the rule we tell the model match the rule we measure?** Under strict, the honest version of the experiment puts the broader rule in the prompt — “no `for`, no `while`, no recursion, no `map`, no `reduce`, no `itertools`, no `sum/any/all/min/max` over a generator, no `next`.” That is a different experiment from the one we ran for Phases 1–2, and the existing numbers do not transfer to it.

5 One-paragraph summary

The training pipeline works, the rationale-SFT result is real, the next experiment is queued and waiting. The blocker is conceptual rather than technical. The rule that defines the whole project admits a permissive reading under which roughly 95% of the benchmark is trivially compliant (and therefore the rule is barely studying anything), and a faithful reading that is broader than what the model was told and unavoidably involves judgment calls. The project needs to commit to one of these two readings before the next launch — or commit to the harder thing, publishing both and treating the gap between them as the finding.